

ADVANCED PROGRAMMING

Assignment Report

Assignment 17

Name	Aritra Talukdar
Roll Number	CSB24034
Course	Advanced Programming
Assignment Number	17
Topic	User Onboarding Validation Module using Python
Programming Language	Python
Testing Framework	Pytest
IDE Used	Visual Studio Code

Department of Computer Science and Engineering

School of Engineering

Tezpur University

May 19, 2026

Contents

1	Introduction	2
2	Objectives	2
3	System Requirements	2
4	Theoretical Background	3
4.1	Custom Exceptions	3
4.2	Regular Expressions (Regex)	3
4.3	Assertions	3
4.4	Pytest Framework	4
5	Program Design	4
6	Algorithm	4
7	Flow of Execution	5
8	Python Source Code	5
8.1	Main Program	5
8.2	Pytest Test Suite	9
9	Output	12
9.1	Successful Registration Output	12
9.2	Invalid Email Output	13
9.3	Underage User Output	14
9.4	Pytest Execution Output	15
10	Observations	16
11	Advantages	16
12	Limitations	16
13	Future Enhancements	16
14	Conclusion	17

1. Introduction

The objective of this assignment is to design and implement a **User Onboarding Validation Module** using Python.

The system validates user registration data before allowing a user to create an account on the platform. It verifies whether the email address is correctly formatted and checks whether the user satisfies the minimum age requirement.

This assignment demonstrates practical implementation of:

- Custom Exception Handling
- Regular Expression Based Validation
- Assertions in Python
- Object Oriented Programming Concepts
- Automated Unit Testing using Pytest

The project ensures data reliability and improves the overall robustness of the registration system.

2. Objectives

The objectives of this assignment are:

- To implement custom exception handling in Python.
- To validate email addresses using Regular Expressions.
- To enforce age restrictions during user registration.
- To understand assertion statements and invariants.
- To automate software testing using Pytest.
- To improve software reliability and maintainability.

3. System Requirements

Requirement	Details
Programming Language	Python 3.x
Operating System	Windows/Linux/macOS

IDE Used	Visual Studio Code
Testing Framework	Pytest
Python Modules Used	re, pytest

4. Theoretical Background

4.1. Custom Exceptions

Custom exceptions are user-defined exceptions created to handle specific error conditions within a program.

In this assignment:

- `InvalidEmailError` is raised when an invalid email is entered.
- `UnderageError` is raised when the user age is below 18.

Custom exceptions improve readability and provide meaningful error handling mechanisms.

4.2. Regular Expressions (Regex)

Regular Expressions are patterns used for string matching and validation.

The regex pattern used in this assignment is:

```
^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$
```

This pattern validates:

- Username section
- @ symbol
- Domain name
- Valid domain extension

4.3. Assertions

Assertions are debugging statements used to verify assumptions within a program.

Example:

```
assert age >= 0
```

If the condition becomes false, Python raises an `AssertionError`.

Assertions are useful for detecting invalid system states.

4.4. Pytest Framework

Pytest is a testing framework used for automated testing in Python.

Features used in this assignment:

- `@pytest.fixture`
- `pytest.raises`
- Automated Test Execution

Pytest helps verify program correctness efficiently.

5. Program Design

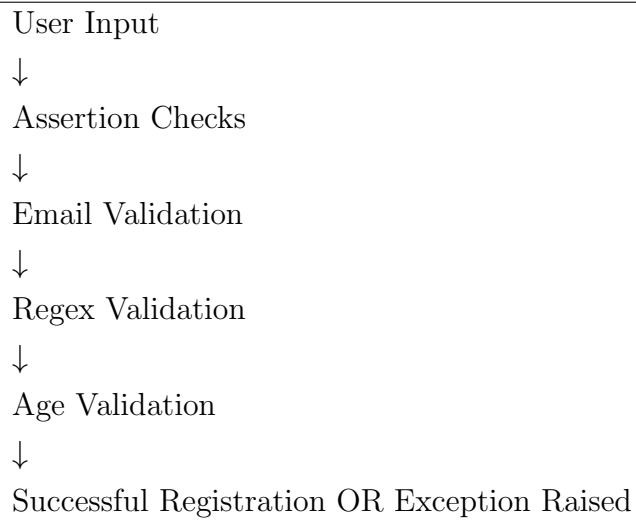
The system consists of:

- Custom Exception Classes
- Registration Service Class
- Email Validation Logic
- Age Validation Logic
- Automated Unit Test Suite

6. Algorithm

1. Start the program.
2. Accept email and age from the user.
3. Perform assertion checks.
4. Validate whether the email is empty.
5. Validate email format using Regular Expressions.
6. Check whether age is at least 18.
7. If validation fails:
 - Raise appropriate custom exception.
8. Otherwise:
 - Display successful registration message.
9. End the program.

7. Flow of Execution



8. Python Source Code

8.1. Main Program

```
"""
=====
          USER ONBOARDING VALIDATION MODULE
=====

This module validates:
1. User Email
2. User Age

Author : Student Assignment
Language : Python
=====
"""

# Importing regular expression module
import re

# =====
# CUSTOM EXCEPTION CLASSES
# =====
```

```
class InvalidEmailError(ValueError):
    """
    Raised when the email is invalid.
    Inherits from ValueError because
    the input value is incorrect.
    """

    def __init__(self, email):
        message = (
            f"\n[EMAIL VALIDATION FAILED]\n"
            f"Provided email '{email}' is invalid.\n"
            f>Please enter a valid email address.\n"
        )
        super().__init__(message)

class UnderageError(Exception):
    """
    Raised when the user's age is below 18.
    """

    def __init__(self, age):
        message = (
            f"\n[AGE VALIDATION FAILED]\n"
            f"Registration denied.\n"
            f"Provided age: {age}\n"
            f"Minimum required age: 18\n"
        )
        super().__init__(message)

# =====
# REGISTRATION SERVICE CLASS
# =====

class RegistrationService:

    def __init__(self):
        """
        Constructor initializes regex pattern.
        """
```

```

# Regular expression for email validation
self.email_pattern = (
    r'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'
)

# -----
# METHOD : register_user
# -----

def register_user(self, email: str, age: int) -> bool:
    """
    Validates user registration details.

    Parameters:
        email (str): User email address
        age (int): User age

    Returns:
        bool: True if registration succeeds

    Raises:
        InvalidEmailError
        UnderageError
    """

    # =====
    # ASSERTION CHECKS (Internal Invariant Checks)
    # =====

    # Age should never be negative
    assert age >= 0, "System Error: Age cannot be negative."

    # Email object should not be None
    assert email is not None, "System Error: Email is None."

    # =====
    # EMAIL VALIDATION
    # =====

    # Remove leading/trailing spaces

```

```

        email = email.strip()

        # Check if email is empty
        if email == "":
            raise InvalidEmailError(email)

        # Regex validation
        if not re.match(self.email_pattern, email):
            raise InvalidEmailError(email)

        # =====
        # AGE VALIDATION
        # =====

        if age < 18:
            raise UnderageError(age)

        # =====
        # SUCCESSFUL REGISTRATION
        # =====

        print("\n
            =====")
        print("          REGISTRATION SUCCESSFUL")
        print("===== "
            )
        print(f"Registered Email : {email}")
        print(f"Registered Age   : {age}")
        print("Account Status   : ACTIVE")
        print("===== \n
            n")

        return True

# =====
# DRIVER CODE
# =====

if __name__ == "__main__":

```

```

service = RegistrationService()

print("\n=====")
print("      USER REGISTRATION VALIDATION SYSTEM")
print("=====")

try:

    email = input("\nEnter Email : ")
    age = int(input("Enter Age : "))

    service.register_user(email, age)

except InvalidEmailError as e:
    print(e)

except UnderageError as e:
    print(e)

except AssertionError as e:
    print(f"\n[ASSERTION ERROR]\n{e}")

except ValueError:
    print("\nInvalid input. Age must be numeric.")

finally:
    print("\nProgram execution completed.\n")

```

8.2. Pytest Test Suite

```

"""
=====
      PYTEST UNIT TEST SUITE
=====
"""

import pytest

from registration_service import (
    RegistrationService,

```

```
        InvalidEmailError,
        UnderageError
    )

# =====
# SHARED FIXTURE
# =====

@pytest.fixture
def service():
    """
    Creates a shared RegistrationService object
    before every test.
    """

    return RegistrationService()

# =====
# TEST : SUCCESSFUL REGISTRATION
# =====

def test_valid_registration(service):

    result = service.register_user(
        "student123@gmail.com",
        20
    )

    assert result is True

# =====
# TEST : EMPTY EMAIL
# =====

def test_empty_email(service):

    with pytest.raises(InvalidEmailError):
```

```
        service.register_user("", 20)

# =====
# TEST : INVALID EMAIL FORMAT
# =====

def test_invalid_email(service):

    with pytest.raises(InvalidEmailError):

        service.register_user("invalid-email", 20)

# =====
# TEST : UNDERAGE USER
# =====

def test_underage_user(service):

    with pytest.raises(UnderageError):

        service.register_user("abc@gmail.com", 15)

# =====
# TEST : NEGATIVE AGE ASSERTION
# =====

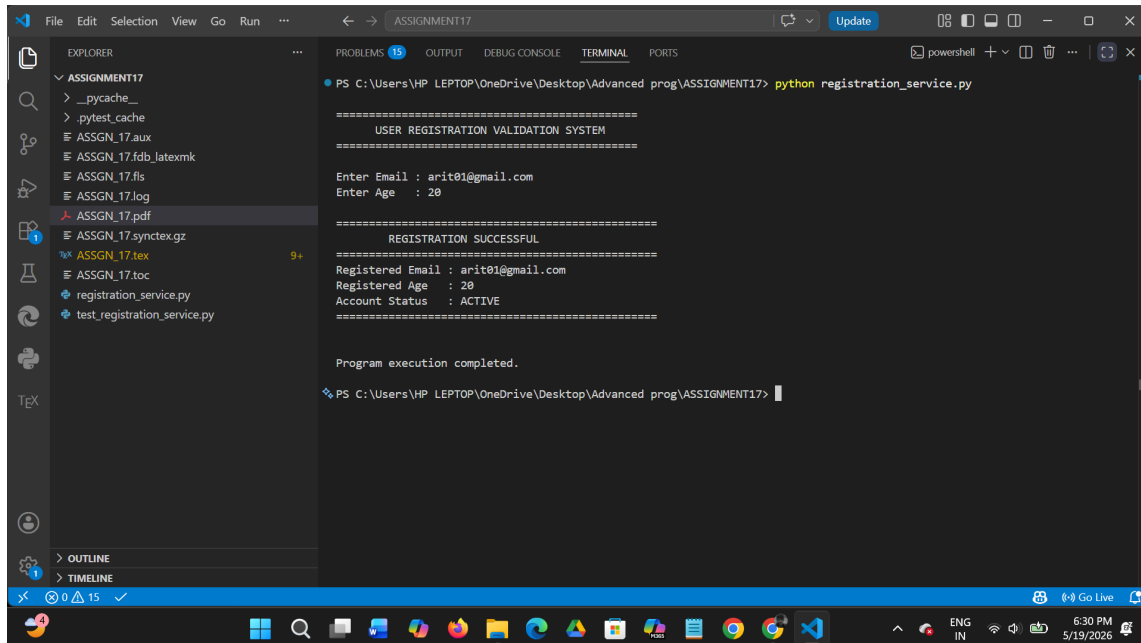
def test_negative_age_assertion(service):

    with pytest.raises(AssertionError):

        service.register_user("abc@gmail.com", -5)
```

9. Output

9.1. Successful Registration Output



The screenshot shows a Visual Studio Code interface with a terminal window open. The terminal displays the output of a Python script named `registration_service.py`. The script prompts for an email and age, and then displays the registration details.

```
PS C:\Users\HP LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT17> python registration_service.py

=====
USER REGISTRATION VALIDATION SYSTEM
=====

Enter Email : arit01@gmail.com
Enter Age   : 20

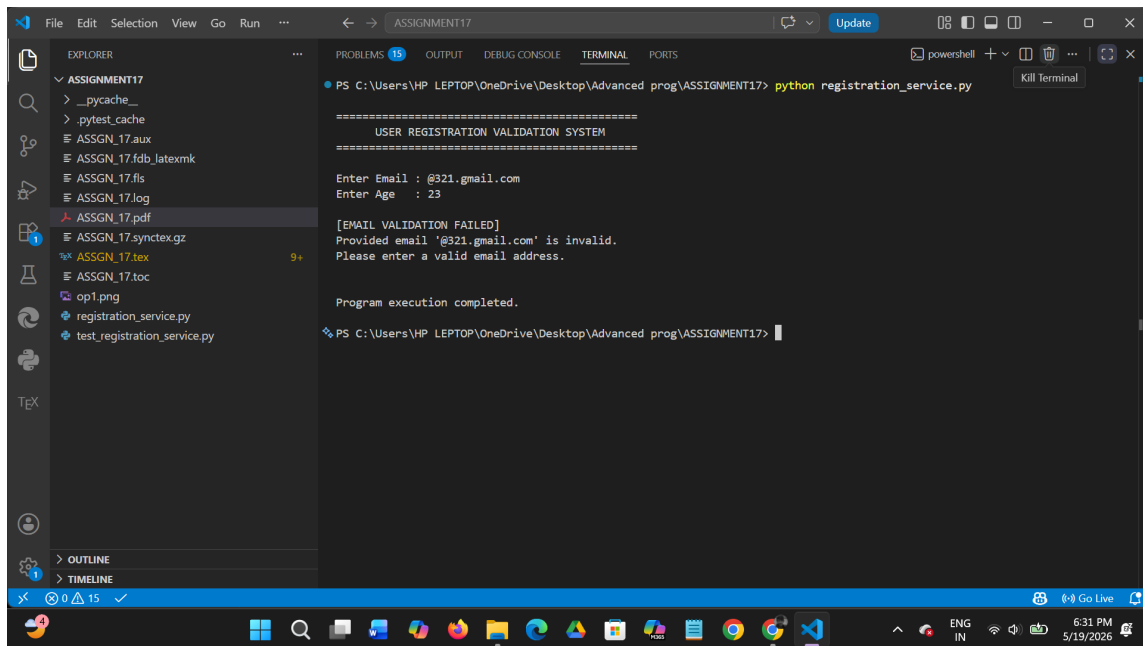
=====
REGISTRATION SUCCESSFUL
=====
Registered Email : arit01@gmail.com
Registered Age   : 20
Account Status   : ACTIVE

Program execution completed.

PS C:\Users\HP LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT17>
```

Figure 1: Successful Registration Output

9.2. Invalid Email Output



```
File Edit Selection View Go Run ... ASSIGNMENT17
EXPLORER
  ASSIGNMENT17
    > __pycache__
    > .pytest_cache
    ASGN_17.aux
    ASGN_17.fdb_latexmk
    ASGN_17.flis
    ASGN_17.log
    ASGN_17.pdf
    ASGN_17.synctex.gz
    ASGN_17.tex
    ASGN_17.toc
    op1.png
    registration_service.py
    test_registration_service.py
  TeX
  OUTLINE
  TIMELINE
  0 15 ✓
  ENG IN 6:31 PM 5/19/2026

PS C:\Users\HP LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT17> python registration_service.py

=====
USER REGISTRATION VALIDATION SYSTEM
=====

Enter Email : @321.gmail.com
Enter Age   : 23

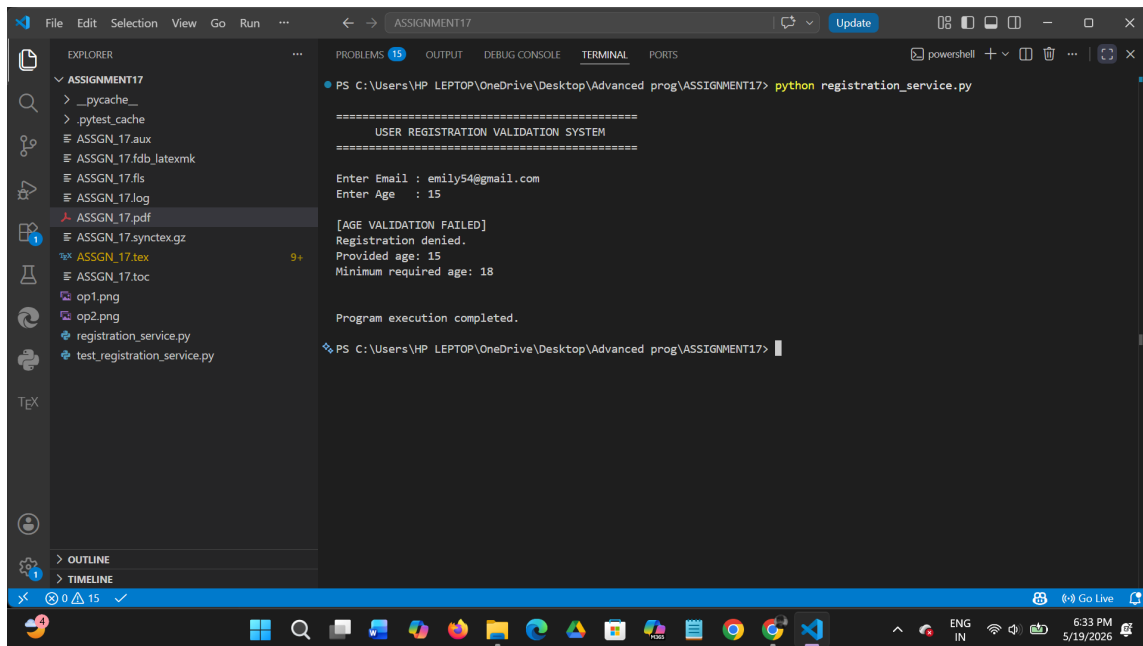
[EMAIL VALIDATION FAILED]
Provided email '@321.gmail.com' is invalid.
Please enter a valid email address.

Program execution completed.

PS C:\Users\HP LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT17>
```

Figure 2: Invalid Email Validation Output

9.3. Underage User Output



```
File Edit Selection View Go Run ... ASSIGNMENT17 Update
PROBLEMS 15 OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + -
EXPLORER
  ASSIGNMENT17
  > __pycache__
  > .pytest_cache
  ASSGN_17.aux
  ASSGN_17.fdb_latexmk
  ASSGN_17.fls
  ASSGN_17.log
  ASSGN_17.pdf
  ASSGN_17.synctex.gz
  ASSGN_17.tex 9+
  ASSGN_17.toc
  op1.png
  op2.png
  registration_service.py
  test_registration_service.py
  TeX
  > OUTLINE
  > TIMELINE
  x 0 15 ✓
  ENG IN 6:33 PM 5/19/2026

PS C:\Users\HP LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT17> python registration_service.py

=====
USER REGISTRATION VALIDATION SYSTEM
=====

Enter Email : emily54@gmail.com
Enter Age   : 15

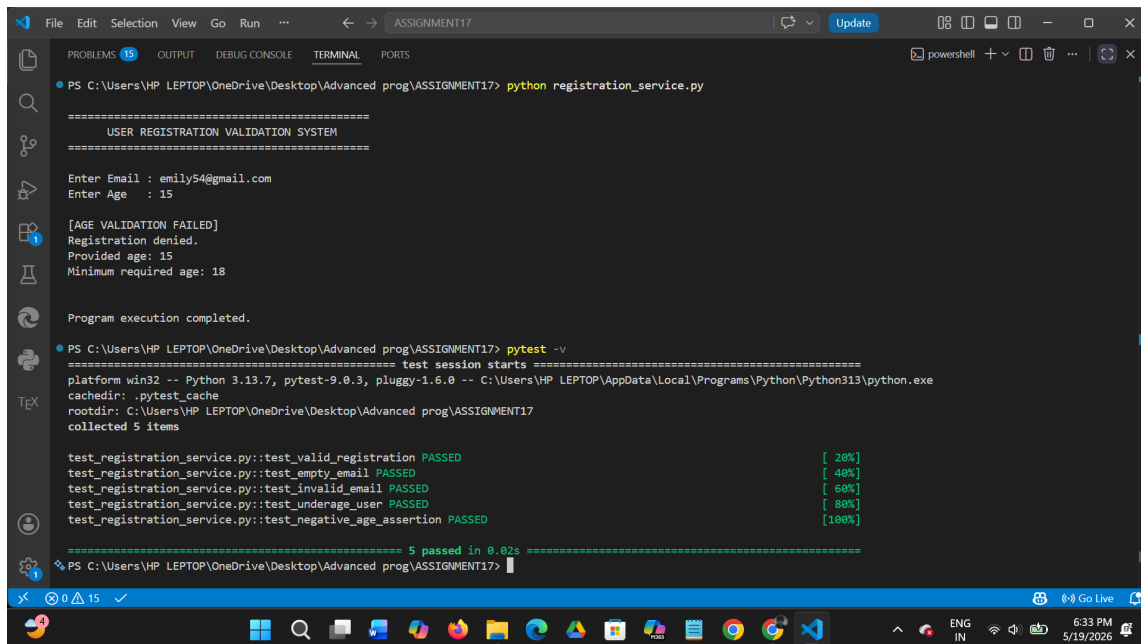
[AGE VALIDATION FAILED]
Registration denied.
Provided age: 15
Minimum required age: 18

Program execution completed.

PS C:\Users\HP LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT17>
```

Figure 3: Underage User Validation Output

9.4. Pytest Execution Output



The screenshot shows a Windows PowerShell terminal window with the following content:

```
PS C:\Users\HP_LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT17> python registration_service.py

=====
USER REGISTRATION VALIDATION SYSTEM
=====

Enter Email : emily54@gmail.com
Enter Age   : 15

[AGE VALIDATION FAILED]
Registration denied.
Provided age: 15
Minimum required age: 18

Program execution completed.

PS C:\Users\HP_LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT17> pytest -v
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\HP_LEPTOP\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\HP_LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT17
collected 5 items

test_registration_service.py::test_valid_registration PASSED [ 20%]
test_registration_service.py::test_empty_email PASSED [ 40%]
test_registration_service.py::test_invalid_email PASSED [ 60%]
test_registration_service.py::test_underage_user PASSED [ 80%]
test_registration_service.py::test_negative_age_assertion PASSED [100%]

===== 5 passed in 0.02s =====

PS C:\Users\HP_LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT17>
```

Figure 4: Pytest Test Case Execution

10. Observations

- The system correctly validates email addresses.
- Invalid email formats trigger custom exceptions.
- Users below 18 years are restricted from registration.
- Assertions detect invalid system states effectively.
- Pytest successfully validates all test cases.
- Automated testing improves software reliability.

11. Advantages

- Improves data validation accuracy.
- Prevents invalid user registrations.
- Enhances software reliability.
- Simplifies debugging and maintenance.
- Automated testing reduces manual effort.

12. Limitations

- Only basic email validation is implemented.
- Database integration is not included.
- Password validation is not implemented.
- GUI or Web Interface is not implemented.

13. Future Enhancements

Future improvements may include:

- Database connectivity
- Email OTP verification
- Password strength validation
- Web-based user interface

- Multi-user support
- Logging and monitoring systems

14. Conclusion

This assignment successfully implemented a User Onboarding Validation Module using Python.

The system validates email addresses using Regular Expressions, checks minimum age restrictions, and handles invalid cases using custom exceptions.

The implementation also demonstrated the use of assertions and automated testing using the Pytest framework.

All unit test cases executed successfully, proving the correctness, reliability, and robustness of the system.